

TREES — VOCABULARY & BINARY TREE CLASS

"Linked lists with two pointers — and suddenly, everything is hierarchical."



■ TODAY IN THREE BLOCKS ■

Yesterday we wrote recursion on a chain of nodes. Today the chain branches. We split the class into three short blocks — **vocabulary, classes, recursion** — each about ten minutes.

Block	What we cover
1	Vocabulary + real-world examples
2	The Node and BinaryTree classes
3	Recursion on trees — <code>count_nodes</code> , <code>find_max_value</code>

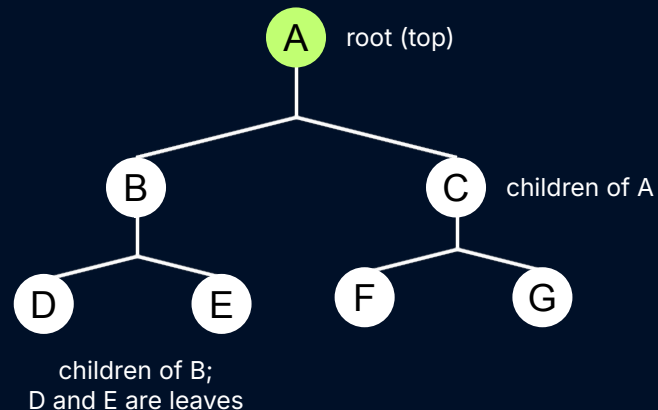
"One extra pointer. One extra recursive call. A whole new world of data."

■ A TREE IS HIERARCHICAL ■

A tree is a hierarchical data structure. Linear structures (arrays, linked lists, stacks, queues) give every element at most one successor. A tree node can have many. The simplest mental picture: an upside-down tree.

Two key differences from a linked list:

- 1) Only one direction (top-down)
- 2) Branching (many children, not just one .next)



■ THE WORDS OF TREES ■

A small set of words. Memorize them — every exam, every project will use them. Each row points back to the diagram on the previous slide so you can see what the word means.

Word	Meaning	On the diagram
Root	The top node. The only node with no parent.	A
Parent	The node directly above another.	A is parent of B and C
Child	The node directly below another.	B and C are children of A
Siblings	Nodes with the same parent.	D and E; B and C
Leaf	A node with no children.	D, E, C
Subtree	A node + everything below it.	Subtree at B = {B, D, E}
Depth (node)	Edges from root to that node.	depth(D) = 2
Height (tree)	Depth of the deepest leaf.	height = 2

Two traps: depth is for a NODE, height is for a TREE. The root has depth 0, not 1.

"Depth is a property of a node. Height is a property of a tree."

■ TREES IN REAL SYSTEMS ■

Why do we care about trees? Because real systems use them constantly. Once you see the pattern, you start spotting trees everywhere — your laptop's files, this web page, the comment thread under any video.

System	What's a node?	Parent → child link
File system	A folder or a file	Folder contains item
HTML / DOM	An HTML element	<body> contains <div> contains <p>
Bilibili comments	A comment	A reply is a child of its parent
Org chart	An employee	Manager → employee
Decision tree	A decision	Branch leads to next decision or leaf
Family tree	A person	Parent → child

Project 1 in five weeks is a Bilibili comment thread modeled as a tree — `add_reply()`, `count_replies()`, `deepest_chain()`. Today is the foundation.

"Once you know the shape, you see trees in every system you use."

■ FROM CHAIN NODE TO TREE NODE ■

A binary tree is a tree where every node has at most two children — **left** and **right**.

The Node class is yesterday's linked list node with one extra pointer. That single change is the only structural difference.

Class 02 (linked list) :

```
class _Node:
    def __init__(self, data):
        self.data = data
        self.next = None # one successor
```

Class 05 (binary tree):

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None # was 'next'
        self.right = None # NEW
```

"Same node. One more pointer. The whole rest of the class follows from this."

■ ROOT + ADD HELPERS ■

The tree itself is a small wrapper. It holds the root and gives us two helpers to attach children. Each helper returns the new node so we can build the tree top-down in one go.

```
class BinaryTree:
    def __init__(self):
        self.root = None # the tree starts empty

    def add_left(self, parent, value):
        """Attach a new node as the LEFT child of parent."""
        new = Node(value)
        parent.left = new
        return new # return so we can chain calls

    def add_right(self, parent, value):
        """Attach a new node as the RIGHT child of parent."""
        new = Node(value)
        parent.right = new
        return new
```

Deliberately minimal API. Real libraries add `'find'`, `'delete'`, balancing — we'll add what we need as we need it.

"Start small. Add methods only when a real exercise asks for them."

■ FIVE LINES, FIVE NODES ■

Now we build the exact tree from slide 4 with five lines of code. Each `add_left` / `add_right` returns the new node, so we can hold onto it and attach children to it on the next line.

```
tree = BinaryTree()
tree.root = Node("A")
b = tree.add_left(tree.root, "B") # A's left child
c = tree.add_right(tree.root, "C") # A's right
d = tree.add_left(b, "D") # B's left child
e = tree.add_right(b, "E") # B's right child
```

[01] Save the return value

We keep `b` because we need it to attach D and E later.

[02] Skip the variable for leaves

`c` is a leaf — we never use it again, so we could even drop the name.

[03] Read top-down

The code reads exactly like the diagram: root, then row 2, then row 3.

"Trees grow downward in nature. They grow downward in code too."

■ THE RECURSION SKELETON RETURNS ■

Last class's recursion rule: base case returns a neutral value, recursive case combines current with `f(smaller)`. On a linked list, "smaller" was `node.next`. On a tree, "smaller" is `node.left` AND `node.right`. One call becomes two. That's the whole change.

```
def f(node):  
    if node is None:  
        return <neutral>  
    return current + f(node.left) + f(node.right)
```

Base case: return neutral. **Recursive case:** current + `f(left)` + `f(right)`.

[01] Same base case

`if node is None: return <neutral>`

[02] Same combination

`current + f(smaller)`

[03] One extra branch

We replace `f(node.next)` with `f(node.left)` and `f(node.right)`.

"Recursion on a tree is recursion on a chain — with one branch to remember."

■ COUNT_NODES ■

Look at yesterday's **length_recursive** and today's **count_nodes** side by side. The base case is identical. The combination is identical. The only new thing is a second recursive call.

Class 04 (linked list) :

```
def length_recursive(node):  
    if node is None:  
        return 0  
    return 1 + length_recursive(node.next)
```

Class 05 (binary tree):

```
def count_nodes(node):  
    if node is None:  
        return 0  
    return 1 + count_nodes(node.left) \  
        + count_nodes(node.right)
```

"Same base. Same +1. One extra recursive call. That's the whole jump."

■ THE CALL TREE ■

Let's run `count_nodes(A)` on the five-node tree. Each call splits into two smaller calls. The leaves hit the base case and return zero. We add one for the current node and unwind back up.

```
count_nodes(A)
= 1 + count_nodes(B) + count_nodes(C)
  |                               |
  |                               └ 1 + count_nodes(None) + count_nodes(None)
  |                               = 1 + 0 + 0 = 1
  |
  |                               └ 1 + count_nodes(D) + count_nodes(E)
  |                               |                               |
  |                               └ 1 + 0 + 0 = 1           └ 1 + 0 + 0 = 1
  |                               = 1 + 1 + 1 = 3
  |
  |                               = 1 + 3 + 1 = 5
```

Five nodes — A, B, C, D, E.

✓ The call stack behaves exactly like `factorial(4)` yesterday: frames go down, base case returns, frames unwind adding back up.

"Frames go down. Base case fires. Frames unwind. Same shape every time."

■ FIND_MAX_VALUE ■

At each node, we compare three values: my own, the max of my left subtree, and the max of my right subtree. Take the biggest. The empty subtree returns `-inf` — a value smaller than anything real, so it always loses.

```
def find_max_value(node):  
    if node is None:  
        return float('-inf') # neutral: smaller than anything real  
    return max(node.value, find_max_value(node.left), find_max_value(node.right))
```

`-inf` is the neutral element for max, just like `0` is for `+` and `1` is for `*`. `max(real, -inf, -inf) = real`

"Pick the right neutral value, and the empty case takes care of itself."

■ RECAP & NEXT CLASS ■

LEFT — Today, in one breath:

- A tree is hierarchical: one root, no cycles, branching downward.
- Vocabulary cold: root, parent, child, sibling, leaf, subtree, depth, height.
- A binary tree node is a linked list node with **one extra pointer**.
- Recursion on a tree = yesterday's skeleton with **two recursive calls**.
- `count_nodes` is `length_recursive`.
`find_max_value` is `find_max`. Reuse, almost line for line.

RIGHT — Class 06 — Tree traversals:

- Four ways to walk every node: **preorder**, **inorder**, **postorder**, **BFS**.
- The first three are tiny variations of `count_nodes` — change the order of three lines, get three different traversals.
- BFS brings back our friend from Class 02: the **queue**.
- Same skeleton. New muscle memory.

"You already know how to walk a tree. Tomorrow we just choose the order."